



Flow Intelligence

A privacy-preserving instrument for measuring developer flow and human–AI collaboration

Rohan Pillay · University of Massachusetts Amherst · rpillay@umass.edu · July 2026

STATUS: BUILT & DEPLOYED — NO DATA COLLECTED (PENDING IRB APPROVAL)

Flow Intelligence is a research instrument — delivered as a lightweight editor extension plus a cloud backend — that studies how developers experience **flow** while collaborating with AI coding tools. It captures **only behavioral and interaction metadata** (counts, sizes, timings, categories) alongside brief self-reports of flow state, and never collects source code, prompt text, file contents, or file paths.

Motivation & research questions

AI coding assistants have changed how software is written, yet we lack good ways to measure their effect on a developer's cognitive state and the *quality* of the human–AI partnership. Traditional output metrics (lines, commits) poorly capture flow. This project instruments the real working environment to ask:

- **RQ1.** Can developer flow be modeled from passively observed behavioral and AI-interaction signals?
- **RQ2.** How does the pattern of human–AI collaboration (e.g. delegation vs. verification) relate to self-reported flow, confidence, and frustration?

What it measures

Behavioral signals (any editor)

- Edit bursts: added/removed *character counts*, change counts, span
- Focus & window switches
- Diagnostics churn (error/warning counts)
- Git commits (count, changed-file count)
- Coarse coding-session boundaries

AI-interaction signals

- Prompt *length* and attachment counts
- AI file edits: *chars added/removed*, file extension
- Tab/completion acceptances
- Tool uses; shell command *class* (e.g. `test`, `git`)
- Verifications (test-run frequency after AI edits)

Ground-truth labels — Experience Sampling (ESM)

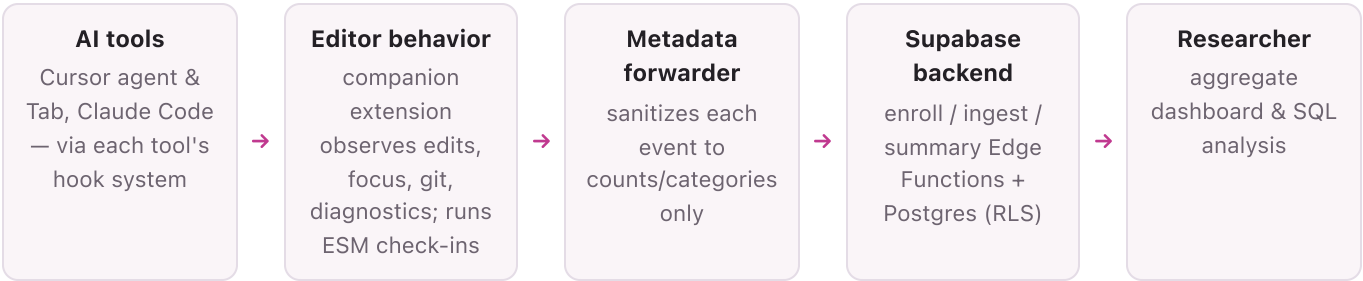
Short, activity-gated in-editor check-ins ask the developer to rate current **flow**, and optionally **frustration** and **confidence**, on 1–5 scales. These self-reports are the target variable for RQ1/RQ2.

Privacy & research-ethics posture

Design principle: “metadata, never content.” No source code, prompt or response text, file contents, file paths, or raw shell commands are ever transmitted or stored — only numeric and short categorical metadata.

- **Two-layer enforcement.** The client sends only derived metadata; the server independently *rejects* any payload containing free-text or forbidden keys (defense-in-depth at the trust boundary).
- **Anonymous by construction.** Participants are identified only by a random UUID and an opaque token; no names, emails, or repository identifiers are stored.
- **Informed consent & enrollment gate.** Data collection begins only after an explicit in-editor consent screen and entry of a study-specific code; a participant cap is enforced.
- **Withdrawal.** A one-click withdraw removes all hooks and halts collection; the server refuses further data from withdrawn participants.
- **Access control.** All database tables enforce Row-Level Security; direct client access is denied. Reads occur only through the researcher's service-role access.

System architecture



The instrument ships as a single extension (published on the Open VSX and Visual Studio Code marketplaces) that installs the appropriate hooks for whichever AI tool the participant uses, and a small serverless backend. No separate runtime install is required by participants.

Platform coverage & signal fidelity

Because no universal API exists to observe every AI tool, the instrument combines direct instrumentation where available with a tool-agnostic behavioral estimate elsewhere. Both are recorded and labeled so analysis can weigh them appropriately.

ENVIRONMENT	AI ATTRIBUTION	BEHAVIORAL & ESM
Cursor — native agent & Tab	Measured (Cursor hooks)	Full
Claude Code (any editor/terminal)	Measured (Claude Code hooks)	Full
VS Code + Copilot / other assistants	Estimated (edit-provenance heuristics)	Full
Any VS Code–based editor	—	Full (behavioral + ESM always work)

Measured attribution comes from exact per-event hooks. **Estimated** attribution infers AI-authored code from large, atomic insertions that don't match human keystroke cadence; raw signals are stored for the analyst to decide precision vs. recall.

Data model

TABLE	CONTENTS
study_codes	Enrollment gate: code, active flag, participant cap.
participants	Anonymous UUID, hashed token, consent version, editor/platform, withdrawal timestamp.
sessions	Coarse coding-session start/end boundaries.
events	Event-sourced metadata (type + JSON payload of counts/categories); tagged by source and agent.
esm_responses	Flow / frustration / confidence self-reports (1–5) — the study labels.

Current status & safeguards

- **Built and deployed:** extension published to Open VSX (Cursor) and the Visual Studio Code Marketplace; backend live on Supabase.
- **No human-subjects data is being collected.** Enrollment is disabled (study code deactivated) and any existing test enrollments are withdrawn, so the system cannot ingest data until a study is deliberately opened.
- Installed-but-unenrolled extensions are completely inert — they transmit nothing without consent and a valid, active code.

What's needed to begin

- **IRB review & approval** of the protocol, consent language, and data-handling plan (all drafted).
- Confirmation of the target **cohort size** and recruitment approach.
- On approval: issue a fresh private study code and re-enable enrollment for consented participants.

Flow Intelligence — project brief. Metadata-only telemetry; anonymized; consent-gated; withdrawable. Prepared for faculty review prior to IRB submission. Details, consent text, and the full signal catalog are available on request.